

TwinRouterBench: 面向真实智能体 LLM 路由的快速静态与在线动态评测

Pei Yang^{1,*} Wanyi Chen^{2,*} Tongyun Yang³ Pengbin Feng⁴ Jiarong Xing⁵ Wentao Guo¹
Yuhang Yao⁶ Yuhang Han⁷ Hanchen Li⁸ Xu Wang⁹ Zeyu Wang¹⁰ Jie Xiao¹ Anjie Yang¹
Liang Tian¹ Lynn Ai¹ Eric Yang¹ Tianyu Shi^{1,†}

¹Gradient ² 苏州大学 ³ 独立研究者 ⁴ 南加州大学 ⁵ 莱斯大学

⁶ 卡内基梅隆大学 ⁷ 上海交通大学

⁸ 加州大学伯克利分校 ⁹ 中国科学院大学 ¹⁰ 加州大学洛杉矶分校

* 共同第一作者 † 通讯作者

在编码智能体、深度研究系统和计算机操作智能体等长时程应用中，LLM 路由尤为重要：一次用户请求会触发多次模型调用。将每次调用路由至能够胜任任务的最低成本模型，可在不牺牲质量的前提下降低成本；但现有路由基准只在一次性提示词上评测路由器。它们既不暴露智能体中间步骤里路由器可见的前缀，也不测试更便宜的替代模型能否保持下游任务成功，且常在评测时依赖在线 LLM 裁判。

我们提出 TwinRouterBench，一个步进路由基准，包含两个赛道。静态赛道提供来自 SWE-bench、BFCL、mtRAG、QMSum 和 PinchBench 的 520 个实例中的 970 个路由器可见前缀；每个前缀都配有按公开的降级-级联协议估计、并经执行验证的目标层级。评分只对层级标签、轨迹归属和 token 成本做确定性算术计算，无需在线的评测端 LLM 裁判。动态赛道提供一个 harness，可在完整的 500 个 SWE-bench Verified 样例上运行路由器；本文报告与静态 SWE 监督集不重叠的 100 个留出样例评测。在每次 LLM 调用时，路由器从锁定的模型池选择一个具体模型；成功由官方任务解决判定和实际 API 支出衡量。两个赛道支持先进行快速的离线迭代，再在在线智能体执行下进行端到端验证。代码和数据见 <https://github.com/CommonstackAI/TwinRouterBench>。

 日期: 2026 年 5 月 8 日

类型: arXiv 预印本

 通讯作者: tianyu@gradient.network

 代码与数据: <https://github.com/CommonstackAI/TwinRouterBench>

1 引言

LLM 路由器决定应由哪个模型处理每一次调用 [Feng et al., 2024, Stripelis et al., 2024]。在生产环境的多轮智能体中，每次调用都带有丰富的前缀——聊天历史、检索段落、shell 日志、工具输出和部分代码修改——而在第 i 步作出的低价选择，可能只会在后续步骤失败时才暴露问题。现有基准在彼此独立的一次性提示词上评测路由；它们既不暴露这些前缀，也不检验某个逐步决策是否会破坏之后的步骤。

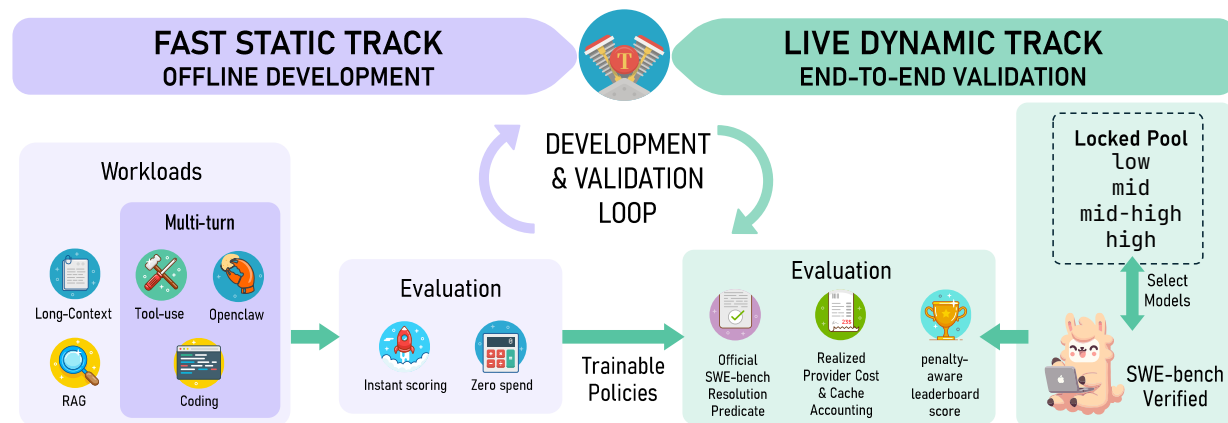


图 1. TwinRouterBench 概览。该基准提供用于离线路由器开发的快速静态赛道，以及用于端到端验证的在线动态赛道。静态赛道覆盖五类工作负载中 520 个实例的 970 条步级记录，每条都带有经执行验证的目标层级；动态赛道在 SWE-bench Verified 上运行路由器，并记录实际 API 成本。

现代应用中昂贵的单元很少是一次性查询。编码智能体通过多轮探索、修改和验证来解决 GitHub issue [Jimenez et al., 2024]；工具系统在串行和并行调用之间维护状态 [Patil et al., 2025]；检索和摘要工作负载携带长的多轮上下文 [Katsis et al., 2025, Zhong et al., 2021]。较便宜的模型或许足以完成文件查找，而后续编写补丁的步骤可能需要最强层级。一个有用的步级基准必须覆盖这些工作负载，以执行结果而非仅依赖模型判断来确定标签，并同时支持低成本、确定性的离线评分与以实际 API 成本为最终验证的端到端执行。因此，我们维护两个彼此区分的赛道。

RouterBench、LLMRouterBench 和 RouterArena 在完整的一次性提示词上评测查询级路由 [Hu et al., 2024, Li et al., 2026, Lu et al., 2025]。此类路由器仍可通过把完整对话历史作为单个查询输入而应用在轨迹中间，但其训练与评测数据均为一次性提示词，所以无法说明在第 i 步使用低价模型会不会破坏后续步骤。我们将每次 LLM 调用前路由器观察到的完整前缀—聊天历史、检索段落、工具输出、shell 轨迹和代码 diff—作为一等输入，并通过将任务运行至结束来验证每个标签：若把某一步降级后，轨迹仍以相同步数通过，我们便推断被替换的步骤已被正确处理。TRIM 研究了数学问题中推理步骤的升级 [Kapoor et al., 2026]；我们的场景则是多轮智能体。

构建这样的基准面临两项主要挑战。第一，将前沿模型直接用作路由器并不可靠：在我们的 Claude Opus 4.6 诊断中，单模板 LLM 路由器在 147 个经验证的 high SWE-bench 步骤中只预测了 7 个 high，并使全部 SWE 轨迹失败。第二，多轮轨迹带来组合式标签复杂度：对于 n 个候选模型和 m 个步骤，即使轨迹长度固定，穷举验证也需要 n^m 次运行，因为第 $i+1$ 步的前缀取决于第 i 步选择的模型。因此，实用的标签构建必须以可负担的协议近似这一空间。

我们提出 TwinRouterBench，一个用于离线开发的快速静态赛道和一个用于端到端验证的在线动态赛道组成的步级路由基准。

贡献。

- **面向真实路由监督的快速静态赛道。**静态赛道覆盖最能从步级路由受益的工作负载（长上下文、多轮、工具使用、RAG、摘要和代码修复），并暴露每次 LLM 调用前路由器所见的精确前缀。其 970 个标签是在固定的降级-级联协议下得到的执行验证估计，包含层级池级联验证、混合前缀重建，以及针对多步可执行工作负载的人工审计。评分只对标签、轨迹归属和 token 成本做确定性算术计算：评测耗时仅毫秒级，且不需要在线评测端 LLM 裁判。这些数据也支持训练；在我们评测的策略中，基于静态标签训练的逻辑回归路由器实现了最佳的成本-成功权衡。
- **面向软件智能体路由的在线动态赛道。**动态 harness 支持完整的 500 个 SWE-bench Verified 样例；本文报告与静态 SWE 监督集不重叠的 100 个留出样例评测。每次 LLM 请求中，路由器从锁定模型池选择一个具体模型；评分使用官方 SWE-bench 解决判定、实际提供商支出、缓存计费，并在排行榜账单中加入固定的未解决实例惩罚 (§5.2)，且没有评测端 LLM 裁判。
- **双赛道开发与验证闭环。**静态赛道提供低成本、可训练的监督以支持快速迭代；动态赛道检验所得策略能否经受真实执行。我们并列报告两者，使离线证据与端到端证据保持可区分。

2 相关工作

查询级路由与级联。 早期研究将模型选择视为对完整查询在质量和成本之间进行的推理时决策。FrugalGPT 学习异构 API 的级联策略以降低成本 [Chen et al., 2024]。AutoMix 先查询较小模型，通过自验证估计可靠性，并在需要时升级 [Aggarwal et al., 2024]。Hybrid LLM 根据预测难度在本地模型与云端模型间路由 [Ding et al., 2024]。RouteLLM 从偏好数据训练路由器，并研究跨模型对的迁移 [Ong et al., 2025]。一篇近期综述将共同目标形式化为在成本预算约束下选择模型 [Varangot-Reille et al., 2025]。这些方法有效地抽象了完整查询上的模型选择，但即便工具输出和局部状态已塑造前缀，多轮智能体轨迹中的中间调用仍缺乏充分研究。

路由器基准。 RouterBench 收集了跨多样化单提示词任务的逾 40.5 万条推理结果，提供了首个用于比较路由算法的大规模测试平台 [Hu et al., 2024]。LLMRouterBench 将规模扩展至逾 40 万个实例、21 个数据集和 33 个模型，并采用统一的性能与成本指标 [Li et al., 2026]；尽管其中包含 500 个 SWE-Bench issue，SWE-Bench 被配置为无智能体脚手架、无中间可路由调用的单查询 0-shot Pass@1 任务。RouterArena 增加了带难度分级和自动排行榜的开放比较平台 [Lu et al., 2025]。Triage 使用代码健康度特征，对每个 SWE-bench Lite issue 作一次层级决策 [Madeyski, 2026]。这些基准对于比较查询级路由器很有价值；但即使纳入 SWE-Bench 等多步任务，路由仍被简化为从预先计算的结果矩阵中为每个 issue 选择一个模型，无法逐一将 issue 内部的中间检索、分析和调试调用路由到每一步足够便宜的模型。

步级路由。 TRIM 通过识别关键推理步骤并将其升级到更大模型，研究多步数学推理的逐步路由；它表明有选择地逐步升级可以在降低平均成本的同时保持准确率 [Kapoor et al., 2026]。但数学推理是相对自包含的场景，缺少真实智能体部署的核心要素：工具输出、检索上下文、shell 轨迹和代码状态。

TRIM 还从假设的推理轨迹导出标签，而未端到端执行降级后的轨迹来验证任务成功。对于可执行的长上下文智能体工作负载，核心标注问题依然存在：在给定当前前缀时，哪个模型层级既足够便宜，又仍能保持最终任务解决？

3 问题形式化与基准概览

3.1 步进路由

一个多轮 LLM 智能体会产生由 LLM 调用构成的轨迹 $\tau = (s_1, s_2, \dots, s_N)$ 。在第 i 步，智能体持有消息历史 $M_i = [m_1, \dots, m_k]$ （即一个前缀：系统提示词、用户指令、先前的助手消息、工具输出和检索片段），产生新的助手消息 $a_i = \mathcal{R}(M_i)$ ，在外部环境中执行 a_i 里的工具调用以得到观察 o_i ，并将 a_i, o_i 追加到历史中： $M_{i+1} = M_i \parallel [a_i, o_i]$ 。当智能体提交结果或预算耗尽时停止。

3.2 条件化的逐调用路由

标准 LLM 路由器将查询映射到候选模型，以优化质量和成本；近期综述将其形式化为在成本预算约束下从模型集合中选择模型 [Varangot-Reille et al., 2025]。TwinRouterBench 研究其条件化的逐调用版本：查询是在下一次 LLM 调用之前完整、路由器可见的前缀 x_i ，其中包括系统消息、用户请求、先前助手消息、工具输出、检索片段、日志和部分代码修改。步进路由器是如下条件决策规则：

$$\pi : x_i \mapsto t_i \in \mathcal{T},$$

其中

$$\mathcal{T} = \{\text{low}, \text{mid}, \text{mid_high}, \text{high}\}$$

是按成本与能力排序的层级集合。每个层级对应一组能力和成本相近的模型；具体模型由下游的层级到模型映射决定，从而使公开记录不包含提供商模型 ID。

理想的条件化目标层级，是在给定前缀 x_i 时能够正确处理当前步骤的最低成本层级。令 $\mathcal{M}_t \subseteq \mathcal{M}$ 表示层级 t 的模型池， $V_i(m; x_i) = 1$ 表示模型 m 在前缀 x_i 下针对当前步骤给出了足够的响应。对于一次性工作负载， V_i 是任务级成功谓词。对于多轮智能体工作负载， $V_i(m; x_i) = 1$ 表示模型 m 正确解决了第 i 步提出的问题；当每一步都被正确解决时，整条轨迹更有可能成功。理想目标为

$$t_i^* = \min \{t \in \mathcal{T} : \exists m \in \mathcal{M}_t \text{ 使得 } V_i(m; x_i) = 1\}.$$

t_i^* 是条件化的逐调用量，而不是关于完整轨迹的全局最优联合策略。部署时，路由器只能观察当前前缀 x_i ：它既不能改变先前输出，也无法在未来调用的前缀出现之前选择未来调用。

由于逐步正确性并非总能孤立判断，我们通过端到端执行来近似 V_i ：当降级步骤后的完整轨迹仍以相同步数通过时，就接受该降级步骤，并据此推断其被正确处理。因此，公开标签 $\hat{t}_i$ 是在固定降级-级联协议 (§4) 下对 t_i^* 的执行验证估计，并由人工审计补充交叉检查该推断 (§4, 人工审计)。当 $\mathcal{M}_i$ 中至少一个模型能解决混合模型轨迹时，即接受层级 t 。我们使用固定的、跨四个层级的 11 模型池；完整列表与分组见附录 (表 A4)。改变模型池、层级成员、harness 或验证协议都会改变 $\hat{t}_i$ ，并构成一个新的基准版本。

3.3 语料库

发布版本包含来自五个基准的 520 条轨迹实例的 970 条步级记录；每个基准的实例数、步骤数和前缀 token 中位数见附录表 A2。一条轨迹是强模型在原始任务上成功完成的一次端到端运行；每一步对应该轨迹内的一次 LLM 调用，其前缀正是路由器在该调用前能看到的消息 (完整构建过程见 §4)。每条记录的字段形状为 id、benchmark、instance_id、step_index、total_steps、messages、target_tier 和 target_tier_id。公开 JSONL 中不含提供商模型 ID，只包含层级标签。完整记录示例见附录。

附录表 A3 汇总了不同层级的成本节约机会分布。low 层级检验路由器能否在保持任务成功的同时避免不必要的昂贵调用。两个中间层级是过渡性的能力带；输出三类的路由器可以将其合并为单一中间带，或通过公开的层级 ID 映射。SWE-bench 贡献了大部分 high 层级记录，表明路由器在困难代码修复步骤中必须保持保守。

mtRAG 和 QMSum 贡献单调用记录，其前缀可能包含多轮上下文。BFCL 同时包含单轮和多轮函数调用：130 个实例产生 248 条记录，其中 29 个实例具有一个以上的路由步骤。SWE-bench 和 PinchBench 贡献多步智能体轨迹。我们纳入所有情形，因为生产路由器会同时遇到单调用的长上下文状态和序列式智能体状态。

3.4 无需在线裁判的静态评测

静态评分只对层级标签、轨迹归属和 token 成本做确定性算术计算；四个指标 RowPass、RowExact、TrajPass 和 CostSave 定义于 §5.1。

3.5 动态 SWE-bench 赛道

动态赛道提供一个支持完整 500 个 SWE-bench Verified 样例的 harness；本文报告的 100 个留出样例与任意静态赛道 SWE-bench 样例均不重叠。每次 LLM 调用时，路由器根据当前前缀、可用模型、缓存状态和预算，从锁定模型池中选择具体模型。该赛道报告三项结果：解决率 (官方 SWE-bench 谓词)、实际 API 支出，以及排行榜账单；后者会在每项策略的 API 成本之上加上相同的固定未解决实例惩罚

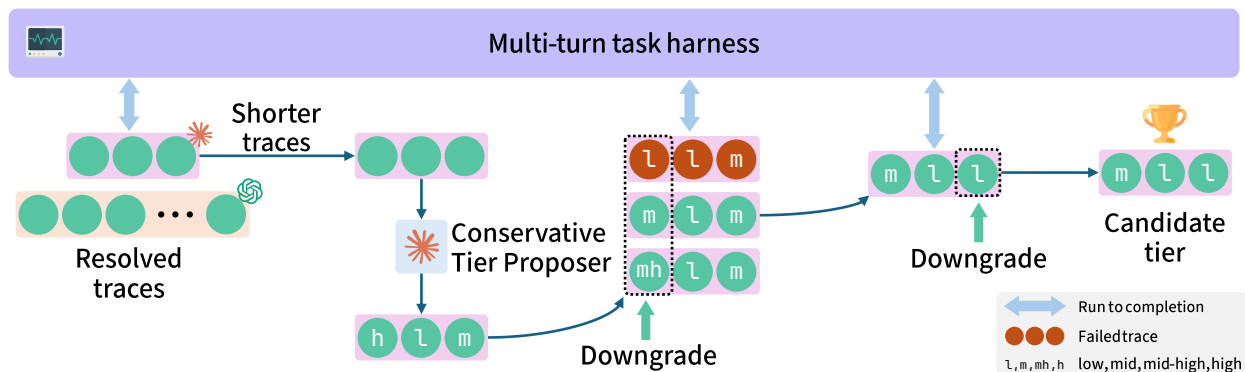


图 2. TwinRouterBench 构建流水线。对于每个多轮样例，流水线从成功的强模型轨迹出发，经由执行验证的搜索将各个步骤逐步降至更低成本的层级，从而为轨迹中的每次 LLM 调用生成已验证的层级标签。

(§5.2)。

4 数据集构建

图 2 展示了如何将每个多轮样例从一条成功的强模型轨迹逐步降级，并通过经执行验证的搜索生成每个步骤的已验证层级标签。

从成功的强模型轨迹开始。 我们首先收集在强模型配置下得到解决的轨迹。对于 SWE-bench，通过 mini-swe-agent 运行的 Claude Opus 4.6 或 GPT-5.4 必须提交能通过官方 FAIL_TO_PASS 测试的补丁；BFCL、mtRAG、QMSum 和 PinchBench [Kilo Code, 2026] 使用各自 harness 的通过谓词。丢弃失败的种子轨迹，以避免路由标签将任务难度与路由错误混为一谈。

在因果前缀下搜索更低层级。 对于每条保留下来的轨迹，Claude Opus 4.6 提供一个搜索提示：该步骤看起来是否可降级，以及若可降级，应首先探测的最激进低层级 h_i 。该提示只是剪枝工具，绝非标签。随后我们运行贪心的顺序锁定降级搜索（附录算法 A1）。在第 i 步，先前锁定的步骤保持其已验证层级，未来步骤使用强层级，并从 h_i 开始，按由低到高的顺序尝试第 i 步的候选。因为逐步正确性并非总能直接观测，只有当完整轨迹仍以相同步数通过时，才接受候选，并推断降级步骤被正确处理；若无候选通过，该步骤保留在强层级。该方法将朴素的 $|T|^N$ 网格降为 $\mathcal{O}(|T|N)$ 次试验，同时确保后续前缀包含先前路由步骤实际产生的降级输出。锁定前缀由以实例、步骤、模型和生成参数为键的响应缓存重放；当上游锁定项改变时，缓存条目失效。

以模型池而非单一模型验证层级。 层级标签应表示该层级的模型池能够解决该步骤，而不应表示某一个模型恰好通过。对于非 low 标签，我们运行三模型级联：只有该层级池中任一模型在混合模型轨迹

基准	总步骤数	抽样步骤数	抽样比例%	可降级	紧致比例%
BFCL	248	25	10.1	0	100.0
SWE-bench	336	34	10.1	1	97.1
PinchBench	48	5	10.4	0	100.0
总计	632	64	10.1	1	98.4

表 1. 步级人工审计汇总。可降级表示审阅者认为已验证层级还可再降低一级而不破坏任务解决；紧致比例表示已处于条件最优层级的比例。我们从具有多步轨迹的三个工作负载中抽取路由步骤；mtRAG 和 QMSum 是已由加固裁判保护的单轮任务，未重新抽样。

下通过，才接受该层级。在 SWE-bench 最后一步审计中，级联在 33 个可降级标签中恢复 28 个，而单模型探测仅恢复 15 个，这证实单一代表模型对监督而言过于脆弱。

加固开放式通过谓词。 一些基准（mtRAG、QMSum）依赖 LLM-as-judge 评测，其默认提示词和评分准则不够严格。在早期开发中，我们人工检查了随机样本的裁判判决，发现表面相似掩盖了错误或不完整答案的伪通过。我们以结构化裁判替换这些弱的参考相似度准则：先检查当前轮是否已解决，再评估忠实性、恰当性和完整性；并对证据冲突设置硬失败、在不确定时保守地失败。我们根据 Radbench 风格的阈值评分校准加固后的准则 [Kuo et al., 2025]，并通过迭代测试验证其消除了初始样本中发现的伪通过模式；结构上不可靠的 mtRAG 样例和所有层级均失败的样例被丢弃。SWE-bench 与 BFCL 已具有可执行或结构化的通过谓词，无需额外加固。

人工审计。 我们的降级搜索固定其余全部步骤，并一次测试一个步骤。该贪心策略将计算成本从指数级降至线性级，但可能遗漏跨步骤的交互效应：两个降级的组合可能失败，即使它们各自单独成功。为防范此类情形，我们对具有多步轨迹的三个工作负载（BFCL、SWE-bench、PinchBench）开展人工抽样审计，从每个工作负载路由步骤中独立随机抽取约 ~10%。mtRAG 和 QMSum 是缺少多轮累积复杂度的单轮任务；其标签已受加固裁判约束，加固过程已通过迭代校准消除了伪通过模式 (§4)，因此未在最终审计中重新抽样。对于每个抽样步骤，审阅者固定路由器可见的前缀，并判断该层级是否还能再降一级，将其标记为紧致、不确定或可继续降级。审计覆盖 64 个步骤（25 个 BFCL、34 个 SWE-bench、5 个 PinchBench）：其中 63 个被判定为紧致；一个 SWE-bench 步骤被标记为可继续降级，且其层级在发布前已下调（表 1）。审计协议细节见附录 A.13。

发布准则。 只有同时满足以下条件的记录才能进入静态赛道：(1) 强模型种子轨迹通过；(2) 层级池中存在一个模型在混合轨迹下通过；(3) 对于开放式工作负载（mtRAG、QMSum），加固裁判不将答案标记为伪通过。每个标签都是能够正确处理当前步骤的最低成本层级的执行验证估计：由于直接的逐步判断并非总是可能，我们从固定步数下的端到端轨迹通过推断逐步充分性，并通过人工审计交叉检查。

5 评测方法、基线与实验设置

5.1 静态评分协议

单条请求的层级标签，无法体现一个 N 步轨迹在第 k 步选错层级所带来的下游成本。我们以四个结合步级视角和轨迹级视角的指标来评估每个路由器：RowPass、RowExact、TrajPass 和 CostSave。前两个按记录计算。TrajPass 要求轨迹中每一个被路由的步骤均通过。CostSave 衡量相对于始终使用 high 基线的节省，但只对通过轨迹记入节省；失败轨迹上的 API 成本被视为已消耗的成本。详细推导见附录。

令 $\hat{t}_i$ 为记录 i 已发布的目标层级， $\tilde{t}_i$ 为路由器预测。当 $\tilde{t}_i \geq \hat{t}_i$ 时，该记录通过；当 $\tilde{t}_i = \hat{t}_i$ 时，该记录精确匹配。只有每条被路由记录都通过时，一条轨迹才通过。令 $c_i(t)$ 为记录 i 在层级 t 下的层级计费成本， T_3 表示 high。对于工作负载 b ，始终使用 high 的账单为 $D_b = \sum_{i \in b} c_i(T_3)$ 。分子 N_b 在轨迹层面累积：通过的轨迹贡献 $\sum_i [c_i(T_3) - c_i(\tilde{t}_i)]$ ；失败的轨迹贡献 $-\sum_i c_i(\tilde{t}_i)$ 。评分器报告

$$\text{CostSave} = \sum_b w_b \frac{N_b}{D_b}, \quad w_b = \frac{n_b}{970},$$

其中 n_b 是工作负载 b 中的记录数，因此在每个成本比率计算后，工作负载按记录数加权。始终使用 high 的策略按构造得到 $\text{CostSave} = 0$ ，因为对于每个工作负载均有 $N_b = 0$ 。

主指标。

$$\text{Combined} = \frac{1}{4}(\text{RowPass} + \text{RowExact} + \text{TrajPass} + \text{CostSave}).$$

这四个分量是主要报告单元；Combined 是在等权重下的一个单一运行点。我们在附录 A.15 中通过消融说明考虑失败的成本计费是合理的。

定价与 token 计费。 high 层级的成本比低层级高 10–50 $\times$ （附录表 A5），因此朴素的逐步成本计费会高估保守路由器。这些是静态的层级常数；动态模型池的模型价格见附录 A.7。提示词 token 被拆分为 cache_read、cache_write 和新输入 input；输出 token 从记录的转录文本中统计。token 计数尽可能使用提供商原生 tokenizer，并以 cl100k_base 作为后备。对于记录 i 和层级 t ，评分器使用与真实 API 计费相同的四桶形式：

$$c_i(t) = \frac{n_i^{\text{in}} p_t^{\text{in}} + n_i^{\text{cr}} p_t^{\text{cr}} + n_i^{\text{cw}} p_t^{\text{cw}} + n_i^{\text{out}} p_t^{\text{out}}}{10^6},$$

其中 token 计数 n_i 和各层级费率 p_i 遵循附录表 A5。静态评分器始终启用提示词缓存建模：在冷启动、层级切换、TTL 到期（5 分钟，与 Anthropic 等提供商的提示词缓存策略默认值一致）或前缀变化时，提示词 token 按 cache_write 计费；在同层级有效前缀命中时按 cache_read 计费；保留新输入项以匹配四桶的动态计费接口。

计入路由特有的影响。 轨迹中途切换模型会带来三类常见担忧：层级切换时的缓存未命中、混合模型历史下的分布外行为，以及不同模型族之间的工具格式不匹配。我们的评估会为每一项计费，而不是假设它们无害。层级切换时的缓存写按进入层级的费率收费（high 的 `cache_write` 是其 `cache_read` 的 12.5 $\times$ ，见附录表 A5），因此频繁切入昂贵层级的策略会按标价承担缓存写成本；动态赛道随后报告实际 OpenRouter 成本而非模拟成本。即便有这些切换，逐步选模型的路由器相对未路由的 Opus 仍将实际 API 成本降低 53.1%（表 3）。混合模型轨迹中的分布外行为由经执行验证的标签 (§4) 和考虑失败的分子处理：未解决轨迹需支付 API 成本及固定惩罚成本，因此 OOD 失败无法表现为节省。工具格式异质性（例如结构化的 `apply_patch` 调用与交错的 `str_replace_editor` 修改）由共享的 `mini-swe-agent harness` 收敛处理；该 harness 在终止时通过 `git diff` 得到每个最终补丁，残余的风格偏差将在同一成本公式下表现为轨迹失败。

5.2 动态 SWE-bench 评分

动态赛道独立于静态代理评分。一次动态运行以 `mini-swe-agent v2.2.8` 作为智能体 harness，端到端执行 SWE-bench Verified 实例；每次 LLM 调用中，路由器从锁定池中选择一个具体模型 ID。逐步成本使用同一四桶公式，但 p 的值是实时的逐模型 OpenRouter 价格（附录 A.7），而 token 桶来自归一化为 `input`、`cache_read`、`cache_write` 和 `output` 的提供商用量日志。令 $A_j = \sum_{i \in \tau_j} c_i(m_i)$ 表示实例 j 的实际 API 成本，其中 m_i 是第 i 步选择的模型， $\text{resolved}_j \in \{0, 1\}$ 遵循官方 SWE-bench 谓词。每个实例的排行榜账单为

$$\text{bill}_j = A_j + (1 - \text{resolved}_j) \gamma,$$

其中 γ 是每个未解决实例固定增加的美元成本（发布默认 $\gamma=0.60$ ，略高于未路由 Opus 4.6 基线观测到的每实例平均 API 成本 \$0.55）。我们将 γ 建模为一个假想完美求解器解决任意 SWE-bench 实例的每样例价格；它统一适用于包括未路由基线在内的所有策略，因此排行榜账单同时反映实际 API 成本和失败后的补救成本。已解决实例只产生 A_j 。排行榜排序键（越低越好）是 $\sum_j \text{bill}_j(\text{total_leaderboard_bill_usd})$ ；`total_router_cost_usd` 和 `total_penalty_cost_usd` 分别报告 API 成本与惩罚成本。作为基础设施失败排除的实例（评分器中的可选标志）不在主结果中附加成本。静态与动态评分从不取平均：静态赛道覆盖五类工作负载，动态赛道只覆盖 SWE-bench Verified。

5.3 基线

我们报告初始的静态参考点：SR-KNN [Liu et al., 2026]，即基于题库嵌入的样本内 1-最近邻上界；ClawRouter [BlockRunAI, 2026]，一个禁用 LLM 后备的开放规则路由器；以及 UncommonRoute [Commonstack AI, 2026]，一个公开的规则路由器，我们将其上游默认配置作为规则基线；下文的训练变体在其接口之上训练。预测均在离线生成，并由公开评分器打分。我们还以 `max_tokens=1` 和提示词缓存运行 Claude Opus 4.6，作为单独的 LLM-as-router 诊断；格式错误的非数字响应被计作错误。

对于训练的 UncommonRoute 变体，我们只训练路由策略，而不训练底层语言模型。最新用户请求由

路由器	RowPass $\uparrow$	RowExact $\uparrow$	TrajPass $\uparrow$	CostSave $\uparrow$	Combined $\uparrow$
SR-KNN (样本内上界)	91.86	78.76	84.74	56.18	77.89
ClawRouter (规则)	80.82	11.75	64.64	53.82	52.76
UncommonRoute (规则)	88.35	61.13	62.37	15.99	56.96
Opus 4.6 (始终 high)	100.00	17.53	100.00	0.00	54.38

表 2. 静态赛道诊断结果 (970 条记录, 均为百分比)。SR-KNN 是样本内上界。Opus 4.6 (始终 high) 行是静态 CostSave 分母所用的 high 层级包络: 每条记录均有 $\hat{t} = \text{high} \geq \hat{t}$, 故 RowPass 和 TrajPass 均为 100%; RowExact 为 170/970 (仅 170 条经验证的 high 记录匹配); CostSave 按构造为 0 (§5.1)。Opus 4.6 作为 LLM-as-router 的结果在 §6.1 单独分析, 因为该运行早于考虑失败的评分公式。

冻结的 BAAI/bge-small-en-v1.5 句子嵌入编码, 再与路由时元数据 (消息数量、是否有工具调用、工具消息数量、请求长度、代码/问题指示符) 拼接。带 L2 正则的多项逻辑回归分类器在训练切分上训练, 正则强度通过 5 折交叉验证选择; 嵌入模型不进行微调。独立的校准切分拟合置信度参数, 并在未用于训练或校准的留出切分上报告性能 (表 3)。附录给出规则配置; 发布包包含锁定的数据与评分代码, 能够复现全部表格。

6 基准结果与分析

6.1 静态赛道诊断结果

表 2 报告静态赛道上全部 970 条记录的分数。SR-KNN [Liu et al., 2026] 是样本内上界参考, 在分隔线以上的三个数据驱动路由器中, 其 Combined 排名第一 (Opus 4.6 (始终 high) 行是 high 层级的成本包络参考, 而非留出监督基线)。动态赛道结果另行报告于 §6.2。

ClawRouter (规则) 的 RowExact 仅为 11.75%, 但 CostSave 为 53.82: 它在大多数非 high 记录上将请求路由至比验证标签高一层的模型, 这破坏了精确匹配, 却仍远比始终 high 便宜。代价是 SWE-bench 上的轨迹失败 (40 条中失败 38 条)。UncommonRoute (规则) 的 RowExact 高得多, 却只有 15.99 CostSave, 因为它在非 high 记录上 217 次跳转到 high, 而且仍使 40 条 SWE 轨迹中的 39 条失败, 令 SWE 切片的 CostSave 降至 -86.08。SR-KNN 取得与 ClawRouter (规则) 相近的 CostSave, 但轨迹保持能力强得多。考虑失败的 CostSave 公式 (§5.1) 确保失败轨迹上的低价调用被收费而非记作收益; 附录 A.15 的消融证实, 更宽松的公式会错误地奖励 ClawRouter (规则) 向上一级的路由。

静态赛道结果按工作负载的分解见附录 A.14。

Opus 作为路由器的案例研究。 我们以描述 11 模型池及其 SWE-bench 解决率的提示词, 在完整基准上运行 Claude Opus 4.6。在 300 条有效的 SWE-bench 记录中, Opus 对 147 个经验证的 high 步骤仅有 7 个预测为 high, 并使全部 40 条 SWE 轨迹失败。这是一个前沿模型在一个提示模板下的案例研究, 而非关于所有 LLM 路由器的普遍结论; 但它说明, 提示前沿模型不能替代经过执行验证的步骤

策略	已解决↑	平均 API 成本↓	API 成本↓	惩罚成本↓	排行榜账单↓
SR-KNN 路由器	75/100	\$0.56	\$55.61	\$15.00	\$70.61
UncommonRoute (训练)	75/100	\$0.26	\$25.66	\$15.00	\$40.66
UncommonRoute (规则)	73/100	\$1.73	\$172.56	\$16.20	\$188.76
Opus 4.6 (无路由)	74/100	\$0.55	\$54.73	\$15.60	\$70.33

表 3. 100 个样例的留出 SWE-bench Verified 切分上的动态赛道结果。API 成本是实际发生的路由 API 支出；惩罚成本为 $\$0.60 \times$ 未解决实例数，其中 $\gamma=0.60$ 是假想完美求解器的每样例价格 (§5.2)；排行榜账单（排序键，越低越好）为 API 成本 + 惩罚成本。该惩罚统一适用于每种策略。未路由的 Opus 4.6 基线位于路由策略下方，与表 2 中的成本包络行相对应。

级监督。完整预测分解见附录 A.11（表 A9）。

6.2 动态留出 SWE 执行

由于每次动态运行都会通过实时 API 调用端到端执行完整的 SWE-bench 智能体，成本随实例数线性增长。为在检验泛化能力的同时控制评测成本，我们随机抽取 100 个不与任何静态赛道 SWE-bench 样例重叠的 SWE-bench Verified 实例，并在这套共享留出集上运行所有策略。表 3 报告结果。各策略的路由监督不同：Opus 和 UncommonRoute（规则）不使用离线路由标签；SR-KNN 使用上游 semantic-router 默认配置 [Liu et al., 2026]；只有 UncommonRoute（训练）在静态赛道标签上拟合 (§5.3)。

在可比的解决率下，训练后的 UncommonRoute 相对未路由 Opus 将 API 成本降低 $1 - 25.66/54.73 = 53.1\%$ ，其成本也比规则变体低 $6.7\times$ （表 3）。因为静态语料库较小，我们通过端到端的动态执行来验证训练后的路由器，而非报告静态留出分数。这提供了初步证据：静态标签能够训练出在这个 100 样例留出切分上维持 SWE-bench 解决率、同时显著降低成本的路由器。

7 结论与局限性

我们发布 TwinRouterBench，一个建立在路由最重要场景之上的步级路由基准：包括 SWE-bench、Pinch-Bench、BFCL、mtRAG 和 QMSum 在内的长上下文、多轮智能体工作负载。静态赛道通过贪心的顺序锁定降级搜索，近似每一步的最低成本充分层级；针对多轮轨迹的人工审计确认了所得标签的可靠性。在动态赛道上，100 样例的留出 SWE-bench 评测证明在线执行框架能产出有意义的端到端结果。一个在静态标签上训练的逻辑回归路由器，在动态赛道上以相同解决率实现 53% 的成本降低，表明静态数据不仅是快速离线评测工具，也是面向实用路由器的有效训练监督。

局限性。 来自 520 个实例的 970 个路由步骤构成的静态语料库，足以进行初步诊断和训练，但并未穷尽全部智能体工作负载、领域或路由模式。标签是在 §5.1 的固定模型池、级联协议和价格前沿下估计的；若未来模型同时变得低价且高能力，就需要更新标签。将动态覆盖扩展到 SWE-bench Verified 之

外，并随模型池和定价演进保持数据最新，是未来工作。

更广泛影响。 通过提供标准化的步级评测，TwinRouterBench 可能加速低成本效益 LLM 路由器的开发，间接降低智能体部署的 API 成本和能源消耗。双赛道设计也将确定性离线评分与在线验证分开，从而改善可复现性。一个潜在负面影响是基准过拟合：社区工作可能集中于当前的 970 条语料和五类工作负载，而牺牲未覆盖的领域、语言或智能体架构。此外，层级标签绑定于固定模型池和价格快照；随着模型与价格演进，陈旧标签可能误导路由器训练或评测。我们通过版本化模型池与定价、明确的范围限制，以及支持更新模型池和重新标注的发布设计来缓解这些风险。

参考文献

- Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A graph-based router for LLM selections, 2024. URL <https://arxiv.org/abs/2410.03834>.
- Dimitris Stripelis, Zhaozhuo Xu, Zijian Hu, Alay Dilipbhai Shah, Han Jin, Yuhang Yao, Jipeng Zhang, Tong Zhang, Salman Avestimehr, and Chaoyang He. TensorOpera router: A multi-model router for efficient LLM inference. In Franck Dernoncourt, Daniel Preotiuc-Pietro, and Anastasia Shimorina, editors, Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pages 452–462, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.34. URL <https://aclanthology.org/2024.emnlp-industry.34/>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): from tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025, volume 267 of Proceedings of Machine Learning Research, pages 48371–48392. PMLR / OpenReview.net, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. mtrag: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems. Trans. Assoc. Comput. Linguistics, 13:784–808, 2025. doi: 10.1162/TACL.A.19. URL <https://doi.org/10.1162/tacl.a.19>.
- Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir R. Radev. QMSum: A new benchmark for query-based multi-domain meeting summarization. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, pages 5905–5921. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.NAACL-MAIN.472. URL <https://doi.org/10.18653/v1/2021.naacl-main.472>.
- Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing

- system. CoRR, abs/2403.12031, 2024. doi: 10.48550/ARXIV.2403.12031. URL <https://doi.org/10.48550/arXiv.2403.12031>.
- Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, et al. LLMRouterBench: A massive benchmark and unified framework for LLM routing. arXiv preprint arXiv:2601.07206, 2026. URL <https://arxiv.org/abs/2601.07206>.
- Yifan Lu, Rixin Liu, Jiayi Yuan, Xingqi Cui, Shenrun Zhang, Hongyi Liu, and Jiarong Xing. Router-Arena: An open platform for comprehensive comparison of LLM routers. CoRR, abs/2510.00202, 2025. doi: 10.48550/ARXIV.2510.00202. URL <https://doi.org/10.48550/arXiv.2510.00202>.
- Vansh Kapoor, Aman Gupta, Hao Chen, Anurag Beniwal, Jing Huang, and Aviral Kumar. TRIM: hybrid inference via targeted stepwise routing in multi-step reasoning tasks. CoRR, abs/2601.10245, 2026. doi: 10.48550/ARXIV.2601.10245. URL <https://doi.org/10.48550/arXiv.2601.10245>.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. Transactions on Machine Learning Research, 2024. URL <https://openreview.net/forum?id=cSimKw5p6R>.
- Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 131000–131034. Curran Associates, Inc., 2024. doi: 10.52202/079017-4164. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ecda225cb187b40ea8edc1f46b03ffda-Paper-Conference.pdf.
- Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: cost-efficient and quality-aware query routing. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL <https://openreview.net/forum?id=02f3mUtqnM>.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL <https://openreview.net/forum?id=8sSqNntaMr>.
- Clovis Varangot-Reille, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. Doing more with less: A survey on routing strategies for resource optimisation in large language model-based systems. CoRR, abs/2502.00409, 2025. doi: 10.48550/ARXIV.2502.00409. URL <https://doi.org/10.48550/arXiv.2502.00409>.

- Lech Madeyski. Triage: Routing software engineering tasks to cost-effective LLM tiers via code quality signals. arXiv preprint arXiv:2604.07494, 2026. URL <https://arxiv.org/abs/2604.07494>.
- Kilo Code. PinchBench: Real-world benchmarks for AI coding agents. <https://github.com/pinchbench/skill>, 2026. Version 2.0.0. MIT license.
- Tzu-Lin Kuo, FengTing Liao, Mu-Wei Hsieh, Fu-Chieh Chang, Po-Chun Hsu, and Da-shan Shiu. RAD-bench: Evaluating large language models’ capabilities in retrieval augmented dialogues. In Weizhu Chen, Yi Yang, Mohammad Kachuee, and Xue-Yong Fu, editors, Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track), pages 868–902, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-194-0. doi: 10.18653/v1/2025.naacl-industry.66. URL <https://aclanthology.org/2025.naacl-industry.66/>.
- Xunzhuo Liu, Huamin Chen, Samzong Lu, Yossi Ovadia, Guohong Wen, Hao Wu, Zhengda Tan, Jintao Zhang, Senan Zedan, Yehudit Kerido, Liav Weiss, Haichen Zhang, Bishen Yu, Asaad Balum, Noa Limoy, Abdallah Samara, Baofa Fan, Brent Salisbury, Ryan Cook, Zhijie Wang, Qiping Pan, Rehan Khan, Avishek Goswami, Houston H. Zhang, Shuyi Wang, Ziang Tang, Fang Han, Zohaib Hassan, Jianqiao Zheng, and Avinash Changrani. vLLM semantic router: Signal driven decision routing for mixture-of-modality models, 2026. URL <https://arxiv.org/abs/2603.04444>. Version 0.3.0. Code: <https://github.com/vllm-project/semantic-router>.
- BlockRunAI. ClawRouter. <https://github.com/BlockRunAI/ClawRouter>, 2026. Version 0.12.149. Accessed: 2026-04-28.
- Commonstack AI. UncommonRoute: An open-source LLM routing library. <https://github.com/CommonstackAI/UncommonRoute>, 2026. Version 0.7.15. Accessed: 2026-04-28.
- Emily M. Bender and Batya Friedman. Data statements for natural language processing: Toward mitigating system bias and enabling better science. Transactions of the Association for Computational Linguistics, 6:587–604, 2018. doi: 10.1162/tacl_a_00041. URL <https://aclanthology.org/Q18-1041/>.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. Communications of the ACM, 64(12): 86–92, 2021. doi: 10.1145/3458723.

A 附录

A.1 数据卡

表 A1. TwinRouterBench 静态赛道的数据卡。它遵循为 LLM 基准发布调整后的 Bender and Friedman [2018] 和 Gebru et al. [2021] 数据声明惯例。

字段	内容
数据集名称	TwinRouterBench v1.0 (静态赛道)
任务类型	步级 LLM 路由 (在层级 ID 0-3 上进行的四分类)
规模	来自 520 个轨迹实例的 970 条记录
源工作负载	SWE-bench (Apache-2.0)、BFCL (CC BY 4.0)、mtRAG (IBM Research; 见原始许可证)、QMSum (MIT)、PinchBench [Kilo Code, 2026] (MIT; 其 53 个任务中 12 个任务派生的 48 条记录被纳入本发布版本, 并受下述包级 Apache-2.0 许可证覆盖)
来源	从强模型成功执行下的智能体轨迹中提取消息前缀; 层级标签由贪心顺序锁定降级搜索和执行验证产生
标签语义	逐步最低成本充分层级的估计: 在 11 模型池、由 Opus 初始化的顺序锁定降级搜索, 以及 §4 的级联协议下, 通过固定步数的端到端轨迹通过进行验证; 即模型池中能够正确处理当前步骤的最低层级, 并通过人工审计交叉检查
许可证	Apache-2.0
预期用途	训练和评测步级 LLM 路由器; 对智能体部署成本降低进行基准评测
范围外用途	宣称绝对路由最优性; 在未重新标注的情况下部署至与发布层级映射显著不同的模型池; 作为通用 SWE-bench 或 BFCL 能力基准使用
潜在偏差	high 层级中过度代表智能体代码修复 (SWE-bench); BFCL/mtRAG/QMSum 在 low 层级近乎饱和; 仅有英文内容; 标签依赖模型池和 harness
维护计划	新的层级池清单和评分协议版本将通过 GitHub releases 跟踪; 层级映射和定价均版本化; 当前发布为 v1.0
人工监督	对 BFCL、SWE-bench 和 PinchBench 的随机步骤样本进行最终人工审计 (总计 64 步, 约 ~10%); 一个 SWE-bench 步骤被标记为可继续降级, 且其验证层级在发布前下调一级; 审计样本中未发现其他仍可降级的标签
发布 URL	公开代码与数据包: https://github.com/CommonstackAI/TwinRouterBench

许可证元数据遵循发布日时的上游文档; 用户在重新分发放派生制品前应核实上游条款。

A.2 语料库概览

基准	实例数	步骤数	每条轨迹步骤数 (中位数)	前缀 token (中位数)	场景
SWE-bench	40	336	9.0	~5,300	GitHub issue 代码修复 (多步智能体)
BFCL	130	248	1.0	~1,600	函数调用 (单轮与多轮)
mtRAG	193	193	1	~1,900	多轮检索增强问答
QMSum	145	145	1	~3,000	面向查询的会议摘要
PinchBench	12	48	4	~10,500	含 23 个任务的通用智能体套件
总计	520	970			

表 A2. 语料库概览。前缀 token 的中位数使用 Anthropic tokenizer 在路由器可见的 messages 字段上计算。

A.3 目标层级分布与顺序锁定搜索

为节省篇幅，我们将工作负载目标层级计数（正文 §3 引用）与顺序锁定降级搜索伪代码（正文 §4 引用）置于本附录。

基准	low	mid	mid_high	high
BFCL	239	8	1	0
mtRAG	183	8	1	1
QMSum	132	10	3	0
PinchBench	41	3	3	1
SWE-bench	94	33	41	168
总计	689	62	49	170

表 A3. 按工作负载划分的目标层级分布。

算法 A1 顺序锁定降级搜索（每条轨迹）

Require: 轨迹 τ 、提示 $h_1, \dots, h_N$ 、harness $\mathcal{H}$ 、层级 $T_0 < T_1 < T_2 < T_3$

- 1: 对所有 i ，令 $\text{locked}[i] \leftarrow T_3$
- 2: for $i = 1$ 到 N do
- 3: if $h_i = \text{不可降级}$ then
- 4: **继续**
- 5: end if
- 6: for t 从 h_i 到 T_2 do
- 7: 运行 $\mathcal{H}$: 步骤 $< i$ 已锁定，步骤 i 位于层级 t ，步骤 $> i$ 位于层级 T_3
- 8: if 轨迹通过 then
- 9: $\text{locked}[i] \leftarrow t$; **终止循环**
- 10: end if
- 11: end for
- 12: end for
- 13: return locked

A.4 制品结构

审稿包包含检查基准和重新计算报告表格所需的静态 JSONL、锁定清单、评分代码、动态切分与紧凑动态摘要。预期布局如下：

```
data/static/question_bank.jsonl
data/static/manifest.json
data/dynamic/model_pool.json
data/dynamic/model_pricing.json
data/dynamic/tier_to_model.json
data/dynamic/ttl_policy.json
main/eval/
main/metrics.py
README.md
```

静态文件包含 970 条记录，单条记录只公开层级标签，不公开提供商模型 ID。动态评分使用 data/dynamic/ 下的锁定模型池、定价、层级映射和 TTL 策略文件；留出的 SWE-bench ID 和聚合输出随发布制品一并提供。README 提供静态评分器和动态评分 CLI 的安装与 smoke-test 命令。对于 E&D 赛道投稿，发布包还包括 metadata/croissant.json，其中含 Croissant 核心字段与 Responsible AI 字段、许可证和源工作负载元数据，以及静态评分器和动态摘要评分器的可执行 smoke-test 脚本。

A.5 静态模型池

静态赛道使用一个固定的、由 11 个具体模型组成的模型池，并将其分为四个能力层级（表 A4）。目标层级标签相对于该模型池作存在性定义：在 §4 的降级-级联过程中，只要层级 t 的模型池中任一个模型能够正确处理当前步骤（依据完整轨迹仍以相同步数通过来推断），便接受该记录属于层级 t 。公开静态记录只存储得到的层级标签，而不存储具体模型 ID，因此当底层模型池更新时，基于这些数据训练的路由器仍保持可移植性。

表 A4. 静态模型池（11 个模型，四个层级）。同一逻辑模型接受的别名在括号中显示。动态赛道代表模型的定价单列于表 A6。

层级	模型
high	anthropic/claude-opus-4.6 (别名: anthropic/claude-opus-4-6) ; openai/gpt-5.4 (别名: openai/gpt-5.4-2026-03-05)
mid_high	anthropic/claude-haiku-4-5; google/gemini-3-flash-preview (别名: gemini/gemini-3-flash-preview); qwen/qwen3.5-397b-a17b
mid	minimax/minimax-m2.5; qwen/qwen3.5-27b; qwen/qwen3-coder
low	deepseek/deepseek-v3.2; z-ai/glm-4.5-air; qwen/qwen3.5-9b

A.6 静态层级计费费率

层级	输入 (\$/1M)	缓存读取 (\$/1M)	缓存写入 (\$/1M)	输出 (\$/1M)
low	0.26	0.13	0.26	0.5
mid	0.30	0.059	0.30	2.0
mid_high	0.50	0.05	0.083	5.0
high	5.0	0.50	6.25	25.0

表 A5. 静态评分协议的逐层级计费常数（美元/百万 token）。这些费率并非任一单独模型的价格，而是从各层级多个模型的价格区间中得出的代表值。high 层级比其余三个层级昂贵 10–50×，后三者之间的价格仅相差 2–5×。动态模型池价格见表 A6。这种不对称驱动附录 A.15 中的消融。

A.7 动态模型池的代表模型与定价

表 A6. TwinRouterBench 动态模型池的具体代表模型及实时 OpenRouter 单价。这些模型价格不同于表 A5 中的静态层级计费常数。价格以美元/百万 token 表示；当提供商未公布单独的缓存写入费率时，缓存写入价格回退为输入价格。这些是动态赛道评分时替入 $c_i(t)$ 的具体 p 值；发布的评分器按包含固定未解决惩罚成本 (§5.2) 的排行榜账单排序。选择 mid 层的代表模型 minimax-m2.7，是因为它在 SWE-bench 排行榜上同等价位模型中排名最高，并与 minimax-m2.5（静态标签构建中使用中层代表）处于相同层级；表 A4 的静态池冻结在构建层级标签时所用的版本。

层级	代表模型	上下文	最大输出	输入	输出	缓存读取	缓存写入
high	anthropic/claude-opus-4.6	1M	128K	5.00	25.00	0.50	6.25
mid_high	google/gemini-3-flash-preview	1.05M	65.5K	0.50	3.00	0.05	0.0833
mid	minimax/minimax-m2.7	196.6K	196.6K	0.30	1.20	0.059	0.30
low	deepseek/deepseek-v3.2	163.8K	163.8K	0.252	0.378	0.0252	0.252

A.8 规则式 UncommonRoute 配置

规则式 UncommonRoute 在双信号集成中使用 MetadataSignal 与 EmbeddingSignal，采用上游默认值 `risk_tolerance=0.5`、`ensemble_weights=(0.55,0.45)` 和 `low_escalation_threshold=0.55`。由于该规则式设置中不存在基准特定的种子集、分类器和 Platt scaler，嵌入信号会弃权，实践中只有元数据信号产生贡献。

A.9 步级成本案例研究：sympy__sympy-12096

表 A7 展示一条 13 步 SWE-bench 轨迹。方案 A 是带提示词缓存的全 high 执行；方案 B 是经验证的步级路由（层级分配见“层级”列）。步级路由在解决该 issue 的同时，相对全 high 实现了 39.6% 的成本降低。最后一步的输出 token 为估计值 (*)。

表 A7. sympy__sympy-12096 的步级成本案例研究。方案 A = 带提示词缓存的全 high。方案 B = 经验证的步级路由。token 数量以千计；成本单位为美元。

步骤	层级	方案 A 输入	方案 A 输出	方案 A 成本	方案 B 输入	方案 B 输出	方案 B 成本
1	mid	1,321	112	\$0.0111	1,284	108	\$0.0005
2	mid	1,744	71	\$0.0051	1,699	68	\$0.0003
3	mid_high	1,846	69	\$0.0032	1,647	66	\$0.0003
4	mid_high	2,502	67	\$0.0067	2,343	65	\$0.0003
5	low	3,287	89	\$0.0084	3,729	87	\$0.0010
6	mid_high	4,103	256	\$0.0131	3,900	254	\$0.0010
7	low	4,512	55	\$0.0060	5,215	55	\$0.0009
8	mid_high	4,612	168	\$0.0071	4,403	168	\$0.0007
9	high	4,921	241	\$0.0103	4,921	241	\$0.0368
10	high	5,149	85	\$0.0060	5,149	85	\$0.0060
11	high	5,591	63	\$0.0069	5,591	63	\$0.0069
12	low	5,653	56	\$0.0046	6,789	59	\$0.0018
13	low	5,864	111*	\$0.0069	7,077	109*	\$0.0010
总计	—	—	—	\$0.0953	—	—	\$0.0576

A.10 mtRAG/QMSum 裁判加固细节

表 A8 记录了从旧版裁判准则到最终数据集所用加固版本的变更。每一行都指明旧版裁判中一种具体的伪通过失败模式，以及构建过程中采用的对应加固规则。

A.11 前沿 LLM-as-Router 诊断汇总

表 A9 展示 Opus 4.6 对经验证的 high SWE-bench 步骤的预测；表 A10 提供额外诊断。这些数值支持如下观点：强模型未经执行的路由判断，不能可靠替代经执行验证的步级监督 (§6.1)。

表 A8. mtRAG/QMSum 裁判加固。这些改动将薄弱的参考相似度判断转化为保守的任务成功谓词, 使其符合当前轮/查询要求和 RadBench 风格的忠实性标准 [Kuo et al., 2025]。

旧版裁判的风险	加固规则	目的
缺乏首要准则	先判定答案是否完整且正确地解决当前轮/查询	防止答案与参考的表面相似替代任务成功
将参考当作答案字符串	将参考视为覆盖提示; 证据/转录文本具有权威性	允许有效改述, 并拒绝盲从参考的错误
无支持事实	对没有支持的关键数字、日期、金额、比例、投票或主张直接硬失败	抑制看似合理的幻觉答案
证据冲突	当候选答案与检索段落或会议转录冲突时直接硬失败	强制忠实性
不可回答/无上下文样例	mtRAG 预过滤: 必须可回答、具有上下文段落且有人类正向相关性反馈	在路由搜索前移除结构上不可靠的样例
所有层级均失败	标记为丢弃; 不生成样例 JSON 或监督记录	防止伪通过成为已验证标签

验证层级	$\tilde{t} = \text{low}$	$\tilde{t} = \text{mid}$	$\tilde{t} = \text{mid_high}$	$\tilde{t} = \text{high}$	总计
$\hat{t} = \text{high}$	34	43	63	7	147

表 A9. Opus 4.6 对经验证 high SWE-bench 步骤的预测。147 个步骤中有 140 个被路由至低于 high 的层级。

A.12 标签验证审计轨迹

表 A11 记录数据集构建期间采用的验证机制。它们共同确保, 除非强模型基线通过、层级池中的模型确实在混合模型轨迹下通过、裁判未将答案标为伪通过, 且最终人工审计未发现遗留问题, 否则没有标签会进入发布的 JSONL。

A.13 人工审计协议与汇总

人工审计是静态赛道的最后一道质量关: 它在由 Opus 初始化的降级搜索、层级池级联检查、开放式裁判加固, 以及 §4 的前缀重建都完成之后执行。审计有意采用人工判断而非第二次 harness 执行。其目的不是重新计算任何通过谓词, 而是在部署路由器也会看到的同一份路由器可见上下文中, 交叉检查已发布层级标签是否仍然站得住脚。

抽样。 我们遵循发布制品中包含的 GT 层级最优性审查协议。审计单元是一条被路由的步骤, 使单步和多步样例在相同粒度上处理。在含有多步轨迹的三类工作负载 (BFCL、SWE-bench、PinchBench) 中, 我们从该工作负载的路由步骤独立随机抽取约 10%。mtRAG 和 QMSum 是缺乏多轮累积复杂度的单轮任务; 其标签已经受到加固裁判的约束, 伪通过模式已通过迭代校准消除 (§4), 因此不在此审计中重新抽样。

表 A10. SWE-bench 上前沿 LLM-as-router (Opus 4.6) 的诊断汇总。核心发现是对 high 层级步骤的严重低估：仅 5% 的经验证 high 步骤被预测为 high，导致该路由下每一条 SWE-bench 轨迹均失败。

诊断项	数值	范围	解释
SWE-bench 有效记录	300	共 336 条记录, 36 条格式错误	用于混淆矩阵分析的有效子集
验证为 high、预测为 high	7/147 (4.8%)	SWE-bench 有效记录	对 high 层级步骤的严重低估
验证为 high、被低估	140/147 (95.2%)	SWE-bench 有效记录	大多数 high 步骤被路由到 low/high 以下的层级
SWE-bench 轨迹通过	0/40	旧版 Opus 路由运行	每条轨迹都有 ≥ 1 个路由不足
中间三分之一位置的低估	73%	步骤位置诊断	核心编辑/测试运行步骤最易被低估
首个三分之一位置的低估	46%	步骤位置诊断	程度较低但仍有显著的路由不足
末个三分之一位置的低估	53%	步骤位置诊断	恢复/收尾步骤仍然困难

表 A11. 标签验证审计轨迹。每个组成部分针对构建流水线中的一种特定失败模式。

验证组件	范围	处理的失败模式	发布操作
成功种子轨迹	全部发布实例	将任务失败与路由失败混淆	只有通过的种子轨迹才进入降级搜索
顺序锁定降级搜索	多步轨迹	漏掉最后一个通过层级; 在最强轨迹上叠加假设性输出	在因果的混合模型前缀下, 从 Opus 建议之下推一层, 直至 harness 失败, 再锁定最后一个通过层级
层级级联 (每层 3 个模型)	全部非 low 层级标签	同一层级中单模型的假阴性	只有一次实际运行通过后才锁定更低成本层级
严格的 mtRAG/QMSum 裁判	开放式 RAG 与摘要	薄弱判定产生的伪通过	使用硬失败准则; 丢弃全部层级失败的样例
最终人工审计	BFCL、SWE-bench 和 PinchBench 各自约 ~10% 的随机步骤样本	上游搜索没有收紧到条件最优的、仍可降级的层级标签	人工审阅在路由器可见前缀下决定发布层级能否再降一级; 一个 SWE-bench 步骤被标记为可继续降级, 并相应下调其层级

审阅流程。 对每个抽样步骤，将路由器可见的 messages 前缀固定为发布版本。可附带一个可选的大模型预标签供参考，但最终决定始终由人工审阅者作出；审阅者判断将已验证层级再降一级后，是否仍会产生正确处理当前步骤的响应。步骤随后标记为紧致、不确定或可继续降级。紧致判定表示发布层级已处于条件最优，无法再降一级而不破坏任务解决；可继续降级判定表示审阅者认为发布层级仍然过高，标签可以再收紧一级。审计总共检查 64 个步骤；一个 SWE-bench 步骤被标记为可继续降级，且其验证层级在发布前下调一级，另 63 个步骤被判定为紧致。这一内部审计旨在捕获上游搜索未收紧至条件最优的残余标签，并澄清标签语义；它并不被表述为外部标注研究，也不是完整语料库的置信区间。

表 A12. 人工审计汇总。当审阅者认为已验证层级还能再降低一级而不破坏任务解决时，抽样步骤被计为可降级；紧致比例是其补集，即层级标签已处于条件最优的比例。抽样在含多步轨迹的三个工作负载上按步骤粒度进行；mtRAG 和 QMSum 是已由加固裁判保护的单轮任务，此处不重新抽样。该审计是有针对性的内部质量检查。

切片	总步骤数	抽样步骤数	抽样比例%	可降级	紧致比例%
BFCL	248	25	10.1	0	100.0
SWE-bench	336	34	10.1	1	97.1
PinchBench	48	5	10.4	0	100.0
总计	632	64	10.1	1	98.4

A.14 静态赛道分解

按工作负载划分的成本节省。 表 A13显示 SWE-bench 是最困难的静态切片。它在按记录加权的 Cost-Save 分数中占 34.64%；由于一个路由不足的步骤就会使整条轨迹失败并触发考虑失败的账单，每个路由器在该切片上的成本节省均为负。对于其余四类工作负载，SR-KNN 和 ClawRouter（规则）平均均超过 85% 的成本节省；全局排名取决于路由器是否保持代码修复轨迹。

路由器	BFCL	mtRAG	QMSum	Pinch	SWE-bench	总体
SR-KNN	90.49	92.35	90.24	35.36	-1.64	56.18
ClawRouter（规则）	89.87	83.45	92.12	55.20	-6.55	53.82
UncommonRoute（规则）	47.14	91.93	90.24	39.80	-86.08	15.99
记录权重	25.57	19.90	14.95	4.95	34.64	100.00

表 A13. 静态评分协议下按工作负载分解的 CostSave。SWE-bench 切片是主要压力测试：失败轨迹会触发 §5.1 中考虑失败的分子。

路由器失败模式。 三个基线以不同方式失败（表 A14）。在 800 条验证层级低于 high 的记录中，SR-KNN 大多数情况下精确匹配；ClawRouter（规则）几乎总是比标签高一层；UncommonRoute（规则）的精确匹配比 ClawRouter（规则）更多，但会 217 次跳至 high。

$\hat{t} < \text{high}$ 时的预测模式	SR-KNN	ClawRouter（规则）	UncommonRoute（规则）
精确层级	627	107	488
高一层	10	649	47
跳至 high	117	21	217
失败：预测低于 $\hat{t}$	46	23	48

表 A14. 800 条非 high 记录上的错误模式。

SWE-bench 说明了为何记录级行为并不充分（表 A15）。UncommonRoute（规则）识别出许多经验证的 high 步骤，但在一条包含 8–13 次调用的轨迹中即使漏掉一个步骤，也会使该实例失败。

A.15 成本节省计费消融

本节给出 §6.1 引用的完整推导和逐变体数值。

路由器	在 $\hat{t} = \text{high}$ 时预测为 high	命中率	失败轨迹
SR-KNN	137/168	81.5%	10/40
ClawRouter (规则)	7/168	4.2%	38/40
UncommonRoute (规则)	105/168	62.5%	39/40

表 A15. SWE-bench 的高层级决策。一个路由不足的关键步骤就会使整条轨迹失败。

定价不对称性驱动消融。 表 A16 针对三个代表性步骤配置，报告各目标层级相对于始终 high 基线的逐步节省。对于三种步骤规模，save_mid / save_low 比值都落在 1 的 3% 以内，这意味着路由器从 $\hat{t} = \text{low}$ 错路由到 $\tilde{t} = \text{mid}$ 在绝对金额上几乎不损失成本。

表 A16. 三个代表性步骤配置上，不同目标层级相对于始终 high 的节省。从 low 错路由到 mid 几乎没有成本。

步骤配置	节省 (high-low)	节省 (high-mid)	节省 (high-mid_high)	save_mid / save_low
4k 冷步骤	3.621c	3.530c	3.467c	0.975
20k 热步骤	1.965c	2.032c	1.900c	1.034
100k 冷步骤	61.13c	60.65c	62.67c	0.992

五种评分变体。 我们在同一组预测上比较五种 N/D 计费方案：

- A (旧版记录级分母)：对满足 $\hat{t}_i \neq \text{high}$ 的通过记录， $D = \sum_i (\text{cost}(3; i) - \text{cost}(\hat{t}_i; i))$ ； N 忽略失败。
- B (旧版全记录分母)： D 与 A 相同，但覆盖全部记录；失败轨迹在轨迹层面减去 $\sum_i \text{cost}(\tilde{t}_i; i)$ 。
- C (中间的 high 记录排除)：从 D 中排除 $\hat{t}_i = \text{high}$ 的记录。
- D ($D = \sum_i \text{cost}(3; i)$ ，在步骤层面惩罚失败，没有轨迹级惩罚)。
- E (最终的考虑失败公式)： D 与 D 相同，但失败轨迹在轨迹层面收取 $-\sum_i \text{cost}(\tilde{t}_i; i)$ 。

表 A17. CostSave 计费消融。变体 A-D 均将 ClawRouter (规则) 排在 SR-KNN 之上；只有变体 E (最终的考虑失败公式) 与 RowPass、RowExact 和 TrajPass 一致。

变体	SR-KNN cost%	ClawRouter (规则) cost%	排名
A (旧版记录级分母)	84.99	89.71	Claw 胜
B (旧版全记录分母)	84.45	84.59	Claw 胜
C (中间 high 记录排除)	79.11	91.31	Claw 胜
D ($D = \sum \text{cost}(3)$ ，步骤级失败)	61.81	67.67	Claw 胜
E (最终考虑失败公式)	56.18	53.82	SR 胜

为什么变体 E 恢复一致的排名。 变体 E 将一条失败轨迹解释为“路由器烧掉了自己的 (低价) 预算，并且用户支付完整 high 账单来重新计算”。形式上，失败轨迹上用户侧账单为 $\sum_i \text{cost}(\tilde{t}_i; i) + \sum_i \text{cost}(3; i)$ ，故相对于始终 high 的节省等于 $-\sum_i \text{cost}(\tilde{t}_i; i)$ ，这正是 N 所编码的值。重试项 $\sum_i \text{cost}(3; i)$ 被 D 吸收而非重复计数，使 $\text{CostSave} \in [-\infty, 100]$ ，其中 0 代表始终 high 的路由。在完整基准上，ClawRouter

(规则) 有 649 个高一层错误 (在绝对金额上近乎免费), 这些错误不再能弥补其 38 条失败的 SWE-bench 轨迹, 因此其 CostSave 与 RowPass/RowExact/TrajPass 数值一致。

A.16 Opus 4.6 混淆矩阵

供参考, 在先前的三指标计费协议下记录的 Opus 4.6 路由运行, 整体分数为 84.05, 其中记录通过 =85.77、记录精确 =82.88、成本节省 =83.48。它不能与表 2 的静态分数直接比较, 因为先前的计费并未使用 §5.1 中考虑失败的轨迹惩罚, 也未报告轨迹通过指标; 因此我们将其视为历史数据点而非用于排名的基线。

表 A18 展示了 §6.1 中汇总的完整混淆矩阵。

表 A18. 300 条有效 SWE-bench 记录 (排除 36 条格式错误输出) 上, Opus 4.6 的路由预测与验证层级。对角单元格加粗。即使验证层级为 high, Opus 也很少选择 high。

	$\tilde{t} = \text{low}$	$\tilde{t} = \text{mid}$	$\tilde{t} = \text{mid_high}$	$\tilde{t} = \text{high}$	行总计
$\hat{t} = \text{low}$	52 (61%)	15	17	1	85
$\hat{t} = \text{mid}$	13	7 (23%)	10	0	30
$\hat{t} = \text{mid_high}$	8	13	16 (42%)	1	38
$\hat{t} = \text{high}$	34	43	63	7 (5%)	147

A.17 mtRAG 记录示例

除 §3 中的 SWE-bench 示例外, 第二种记录格式说明 low 层级标签如何产生: 经过若干次检索轮次后, 一个简短的后续查询可以由模型池中最低成本层级回答。

```
{
  "id": "mtrag_..._turn_3_step_1",
  "benchmark": "mtrag",
  "step_index": 1, "total_steps": 1,
  "messages": [
    {"role": "system", "content": "Answer based on the retrieved passages..."},
    {"role": "user", "content": "[[Passage] Major.minor update... [Passage] ..."},
    {"role": "assistant", "content": "..."},
    ...multi-turn history...
    {"role": "user", "content": "Major.minor update."}
  ],
  "target_tier": "low", "target_tier_id": 0
}
```